



# Testing VPC Connectivity



Arpita Chowdhury

```
(ec2-user@ip-10-0-0-61 ~)$ curl example.com
<!doctype html>
<html>
<head>
  <title>Example Domain</title>

  <meta charset="utf-8" />
  <meta http-equiv="Content-type" content="text/html; charset=utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <style type="text/css">
    body {
      background-color: #f0f0f2;
      margin: 0;
      padding: 0;
      font-family: -apple-system, system-ui, BlinkMacSystemFont, "Segoe UI", "Open Sans", "Helvetica Neue", Helvetica, Arial, sans-serif;
    }
    div {
      width: 600px;
      margin: 5em auto;
      padding: 2em;
      background-color: #fdfdff;
      border-radius: 0.5em;
      box-shadow: 2px 3px 7px rgba(0,0,0,0.02);
    }
    a:link, a:visited {
      color: #38488f;
      text-decoration: none;
    }
    @media (max-width: 700px) {
      div {
        margin: 0 auto;
        width: auto;
      }
    }
  </style>
```



# Introducing Today's Project!

## What is Amazon VPC?

Amazon VPC is a Virtual Private Cloud service that lets you create a logically isolated network in AWS where you can launch and manage your resources securely. It is useful because it gives you full control over your networking environment, including IP address ranges, subnets, route tables, and security settings, enabling you to design secure and scalable architectures in the cloud.

## How I used Amazon VPC in this project

In today's project, I used Amazon VPC to test VPC connectivity.

## One thing I didn't expect in this project was...

One thing I didn't expect in this project was realizing that even with a peering connection, I still need to explicitly update my route tables for both VPCs to direct traffic destined for the peer VPC through the peering connection. This entire process from initial ping failure to understanding and implementing route table updates is a key insight into how VPC peering truly works beyond just establishing the connection itself.



This project took me...

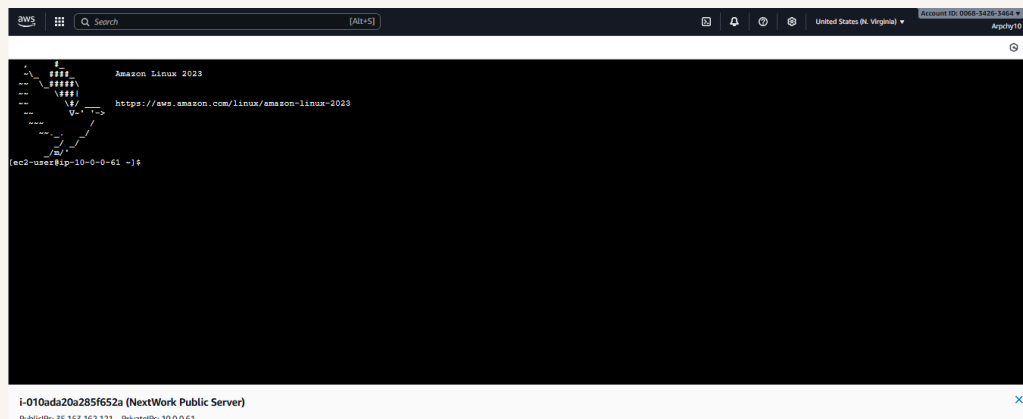
This project took me 2 hours.



# Connecting to an EC2 Instance

Connectivity means the ability of two or more systems, devices, or networks to communicate with each other.

My first connectivity test was whether I could connect to access NextWork Public Server using SSH through EC2 Instance Connect.







## EC2 Instance Connect

I connected to my EC2 instance using EC2 Instance Connect, which is a browser-based SSH client provided by AWS that generates temporary SSH keys and lets you securely access your instance without needing to download or manage a key pair locally.

My first attempt at getting direct access to my public server resulted in an error, because the security group associated with NextWork Public Server lets in all inbound HTTP traffic, but this is not how I am trying to access our Public Server! I am trying to access NextWork Public Server using SSH through EC2 Instance Connect, which is a different traffic type

I fixed this error by updating NextWork Public Server's security group so it can let in SSH traffic. Choosing Anywhere-IPv4 as the source lets in SSH connections from any IPv4 address.





## Connectivity Between Servers

Ping is a network utility that sends ICMP echo requests to a target host to check if it is reachable and measure the response time. I used ping to test the connectivity between two EC2 instances public and private one.

The ping command I ran was `ping 10.0.1.151`.

The first ping returned `PING 10.0.1.151 (10.0.1.151) 56(84) bytes of data`. This meant that my ping request was being sent to the private IP address 10.0.1.151 of the EC2 instance, with each packet carrying 56 bytes of data (84 bytes including the ICMP/IP headers).

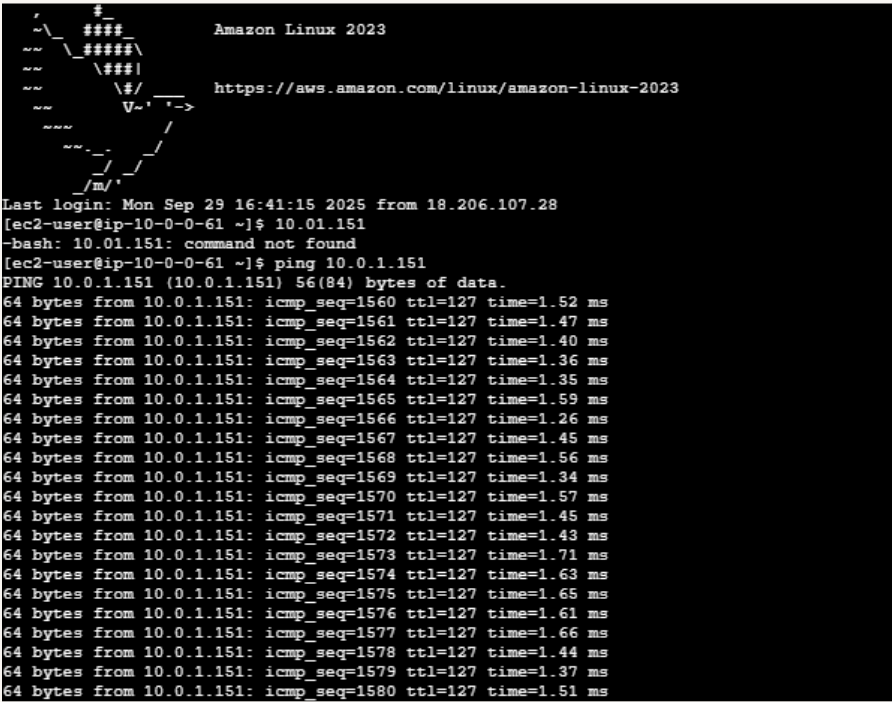
```
#_
~\_ ##### Amazon Linux 2023
~~~\_#####\
~~~\###|
~~~\#/ https://aws.amazon.com/linux/amazon-linux-2023
~~~~V~' ->
~~~~~
~~~~~-
~~~~~/m/'
```

Last login: Mon Sep 29 16:41:15 2025 from 18.206.107.28  
[ec2-user@ip-10-0-0-61 ~]\$ 10.01.151  
-bash: 10.01.151: command not found  
[ec2-user@ip-10-0-0-61 ~]\$ ping 10.0.1.151  
PING 10.0.1.151 (10.0.1.151) 56(84) bytes of data.



## Troubleshooting Connectivity

I troubleshooted this by going to NextWork private subnet and editing the inbound and outbound rule where I added a new rule and gave these variables rule of 100 and type All ICMP - IPv4 and source the NextWork public subnet id and also changed the security group of private NextWork where the inbound rule is edited with the same variables and changes were saved.



```
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

Last login: Mon Sep 29 16:41:15 2025 from 18.206.107.28
[ec2-user@ip-10-0-0-61 ~]$ 10.0.1.151
-bash: 10.0.1.151: command not found
[ec2-user@ip-10-0-0-61 ~]$ ping 10.0.1.151
PING 10.0.1.151 (10.0.1.151) 56(84) bytes of data.
64 bytes from 10.0.1.151: icmp_seq=1560 ttl=127 time=1.52 ms
64 bytes from 10.0.1.151: icmp_seq=1561 ttl=127 time=1.47 ms
64 bytes from 10.0.1.151: icmp_seq=1562 ttl=127 time=1.40 ms
64 bytes from 10.0.1.151: icmp_seq=1563 ttl=127 time=1.36 ms
64 bytes from 10.0.1.151: icmp_seq=1564 ttl=127 time=1.35 ms
64 bytes from 10.0.1.151: icmp_seq=1565 ttl=127 time=1.59 ms
64 bytes from 10.0.1.151: icmp_seq=1566 ttl=127 time=1.26 ms
64 bytes from 10.0.1.151: icmp_seq=1567 ttl=127 time=1.45 ms
64 bytes from 10.0.1.151: icmp_seq=1568 ttl=127 time=1.56 ms
64 bytes from 10.0.1.151: icmp_seq=1569 ttl=127 time=1.34 ms
64 bytes from 10.0.1.151: icmp_seq=1570 ttl=127 time=1.57 ms
64 bytes from 10.0.1.151: icmp_seq=1571 ttl=127 time=1.45 ms
64 bytes from 10.0.1.151: icmp_seq=1572 ttl=127 time=1.43 ms
64 bytes from 10.0.1.151: icmp_seq=1573 ttl=127 time=1.71 ms
64 bytes from 10.0.1.151: icmp_seq=1574 ttl=127 time=1.63 ms
64 bytes from 10.0.1.151: icmp_seq=1575 ttl=127 time=1.65 ms
64 bytes from 10.0.1.151: icmp_seq=1576 ttl=127 time=1.61 ms
64 bytes from 10.0.1.151: icmp_seq=1577 ttl=127 time=1.66 ms
64 bytes from 10.0.1.151: icmp_seq=1578 ttl=127 time=1.44 ms
64 bytes from 10.0.1.151: icmp_seq=1579 ttl=127 time=1.37 ms
64 bytes from 10.0.1.151: icmp_seq=1580 ttl=127 time=1.51 ms
```



# Connectivity to the Internet

Curl is short for “Client URL” is a command-line tool used to transfer data to or from a server using various protocols, most commonly HTTP and HTTPS.

I used curl to test the connectivity between my Public Server instance to see if it could successfully communicate with and retrieve content from the public internet.

## Ping vs Curl

Ping and curl are different because ping tests basic network connectivity by sending ICMP echo requests to check if a host is reachable, while curl tests application-level connectivity by sending HTTP/HTTPS (or other protocol) requests to check if a specific service or webpage is responding.



# Connectivity to the Internet

I ran the curl command `curl example.com`, `curl learn.nextwork.org` and `curl https://learn.nextwork.org/projects/aws-host-a-website-on-s3` which returned raw HTML content from the server.

```
[ec2-user@ip-10-0-0-61 ~]$ curl example.com
<!doctype html>
<html>
<head>
  <title>Example Domain</title>

  <meta charset="utf-8" />
  <meta http-equiv="Content-type" content="text/html; charset=utf-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1" />
  <style type="text/css">
    body {
      background-color: #f0f0f2;
      margin: 0;
      padding: 0;
      font-family: -apple-system, system-ui, BlinkMacSystemFont, "Segoe UI", "Open Sans", "Helvetica Neue", Helvetica, Arial, sans-serif;
    }
    div {
      width: 600px;
      margin: 5em auto;
      padding: 2em;
      background-color: #fdfdff;
      border-radius: 0.5em;
      box-shadow: 2px 3px 7px 2px rgba(0,0,0,0.02);
    }
    a:link, a:visited {
      color: #38488f;
      text-decoration: none;
    }
    @media (max-width: 700px) {
      div {
        margin: 0 auto;
        width: auto;
      }
    }
  </style>
```



[nextwork.org](https://nextwork.org)

# The place to learn & showcase your skills

Check out [nextwork.org](https://nextwork.org) for more projects

